

Boot Loader Project Report

Building a Boot Loader From Scratch

A hands-on exploration of the x86 boot process, from a raw Master Boot Record to a GRUB-loaded kernel.

Contents

1	Introduction	2
2	Prerequisites	2
3	Part 1 — Master Boot Record	2
3.1	Goal	2
3.2	Memory Layout	2
3.3	Source Code: <code>boot.asm</code>	4
3.4	Code Notes	4
3.5	Build and Run	5
3.6	Result	5
4	Part 2 — GRUB and a Minimal Kernel	6
4.1	Goal	6
4.2	File 1 — <code>multiboot_header.asm</code>	6
4.3	File 2 — <code>kernel.asm</code>	6
4.4	File 3 — <code>linker.ld</code>	7
4.5	File 4 — <code>grub.cfg</code>	8
4.6	Building the ISO Image	8
4.7	Boot in QEMU	9
4.8	Result	10
5	Summary	12
	Acknowledgements	12
A	All Commands at a Glance	12

1 Introduction

This report documents a personal project exploring how a computer boots from scratch. The project is a hands-on exercise in which a virtual machine (QEMU) is booted from scratch using two approaches. In Part 1, a raw 512-byte Master Boot Record (MBR) is loaded directly by the BIOS and prints **Hello** to the screen. In Part 2, a GRUB boot loader loads a multiboot-compliant kernel that writes **OK** to the screen via direct VGA memory access.

The objective is to trace the full chain **BIOS** → **MBR** → **Boot Loader** → **Kernel** and confirm that the boot process is a well-defined sequence of conventions.

2 Prerequisites

All required tools were installed prior to starting the project:

```
sudo dnf install nasm qemu-system-x86 grub2-tools-extra xorriso
    mtools
```

3 Part 1 — Master Boot Record

3.1 Goal

Write a 512-byte x86 assembly program that the BIOS loads at address **0x7C00** and executes in 16-bit real mode. The program prints **Hello** using BIOS interrupt **0x10**, then halts.

3.2 Memory Layout

Figure 1 shows the physical memory layout during MBR execution. The stack is placed immediately above the 512-byte MBR body.

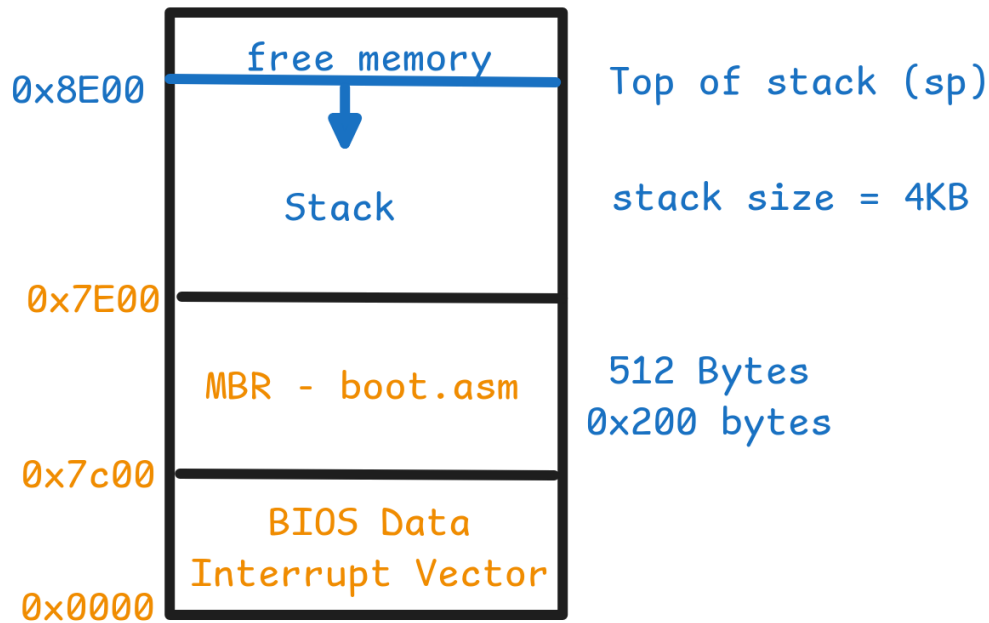


Figure 1: Physical memory layout during MBR execution. The stack base (0x07E00) is placed immediately after the MBR (0x07C00–0x07DFF). The stack pointer starts at offset 0x1000 above the stack base, giving a 4 KiB stack that grows downward.

Register	Value set in code	Physical address
DS	0x07C0	$0x07C0 \times 16 = 0x07C00$
SS	0x07E0	$0x07E0 \times 16 = 0x07E00$
SP	0x1000	$0x07E00 + 0x1000 = 0x08E00$
MBR end	—	$0x07C00 + 511 = 0x07DFF$

3.3 Source Code: boot.asm

```
1 bits 16
2
3 start:
4     mov ax, 0x07C0
5     add ax, 0x20
6     mov ss, ax
7     mov sp, 0x1000
8     mov ax, 0x07C0
9     mov ds, ax
10
11     mov si, msg
12     mov ah, 0x0E
13
14 .next:
15     lodsb
16     cmp al, 0
17     je .done
18     int 0x10
19     jmp .next
20
21 .done:
22     jmp $
23
24 msg:
25     db 'Hello', 0
26
27 times 510-($-$$) db 0
28 dw 0xAA55
```

Listing 1: boot.asm — Master Boot Record

3.4 Code Notes

- **bits 16:** instructs NASM to emit 16-bit opcodes. Every x86 CPU enters 16-bit real mode on power-up.
- **Segment addressing:** physical address = segment $\times$ 16 + offset. Setting DS = 0x07C0 ensures data references resolve relative to 0x07C00 where the BIOS loaded the MBR.
- **lodsb:** reads [DS:SI] into AL and increments SI — convenient for null-terminated strings.
- **Boot signature:** bytes 510–511 must equal 0x55, 0xAA; the BIOS does not treat a

sector as bootable without this.

3.5 Build and Run

```
nasm -f bin -o boot.bin boot.asm
ls -l boot.bin          # must be exactly 512 bytes
qemu-system-x86_64 -drive file=boot.bin,index=0,media=disk,format=raw
```

Figure 2 shows the assembly and binary verification step.

```
r4g4b@fedora:~/OS/M_P_1/Part1$ ls
boot.asm
r4g4b@fedora:~/OS/M_P_1/Part1$ nasm -f bin -o boot.bin boot.asm # Assemble the assembly code as binary
r4g4b@fedora:~/OS/M_P_1/Part1$ ls
boot.asm boot.bin
r4g4b@fedora:~/OS/M_P_1/Part1$ |
```

Figure 2: Assembling `boot.asm` and verifying the output binary is exactly 512 bytes.

3.6 Result

```
r4g4b@fedora:~/OS/M_P_1/Part1$ nasm -f bin -o boot.bin boot.asm # Assemble the assembly code as binary
r4g4b@fedora:~/OS/M_P_1/Part1$ ls
boot.asm boot.bin
r4g4b@fedora:~/OS/M_P_1/Part1$ qemu-system-x86_64 -drive file=boot.bin,index=0,media=disk,format=raw
```



Figure 3: QEMU: Hello printed by the MBR running in 16-bit real mode.

4 Part 2 — GRUB and a Minimal Kernel

4.1 Goal

Use GRUB as the boot loader so all 16-bit initialisation is handled automatically. GRUB delivers the kernel in 32-bit protected mode; the kernel writes OK directly to the VGA text buffer at 0xB8000.

The pipeline is: assemble two source files → link into an ELF binary → package with GRUB into a bootable ISO.

4.2 File 1 — multiboot_header.asm

GRUB locates a kernel by scanning for a Multiboot 2 header containing a magic number, architecture field, byte length, and checksum (the four fields must sum to zero mod 2^{32}).

```
1 section .multiboot_header
2
3 magic equ 0xe85250d6
4 arch equ 0
5
6 header_start:
7     dd magic
8     dd arch
9     dd header_end - header_start
10    dd 0x100000000 - (magic + arch + (header_end - header_start))
11
12    dw 0
13    dw 0
14    dd 8
15 header_end:
```

Listing 2: multiboot_header.asm

```
nasm -f elf32 multiboot_header.asm # -> multiboot_header.o
```

4.3 File 2 — kernel.asm

The kernel runs in 32-bit protected mode. BIOS calls are unavailable, so output is produced by writing directly to the VGA text buffer. Each character cell is two bytes: low byte = ASCII code, high byte = colour attribute (0x2F = white text on green background).

```

1 bits 32
2 global start
3
4 section .text
5 start:
6     mov dword [0xb8000], 0x2f4b2f4f
7     hlt

```

Listing 3: kernel.asm

```
nasm -f elf32 kernel.asm # -> kernel.o
```

Figure 4 shows the assembled kernel source files.

```

r4g4b@fedora:~/OS/M_P_1/Part2$ ls
kernel.asm linker.ld multiboot_header.asm
r4g4b@fedora:~/OS/M_P_1/Part2$ nasm -f elf32 multiboot_header.asm -o multiboot_header.o
r4g4b@fedora:~/OS/M_P_1/Part2$ ls
kernel.asm linker.ld multiboot_header.asm multiboot_header.o
r4g4b@fedora:~/OS/M_P_1/Part2$ nasm -f elf32 kernel.asm -o kernel.o
r4g4b@fedora:~/OS/M_P_1/Part2$ ls
kernel.asm kernel.o linker.ld multiboot_header.asm multiboot_header.o
r4g4b@fedora:~/OS/M_P_1/Part2$ ld -m elf_i386 -o kernel.bin -T linker.ld multiboot_header.o kernel.o
r4g4b@fedora:~/OS/M_P_1/Part2$ ls
kernel.asm kernel.bin kernel.o linker.ld multiboot_header.asm multiboot_header.o
r4g4b@fedora:~/OS/M_P_1/Part2$

```

Figure 4: Assembled kernel and multiboot header object files.

4.4 File 3 — linker.ld

The linker script joins both object files, places the multiboot header first (GRUB scans only the first 8 KiB), and loads everything at 1 MiB (0x100000) — safely above all BIOS and GRUB reserved memory.

```

1 ENTRY(start)
2
3 SECTIONS {
4     . = 1M;
5
6     .boot : {
7         *(.multiboot_header)
8     }
9
10    .text : {
11        *(.text)
12    }
13 }

```

Listing 4: linker.ld

```
ld -m elf_i386 -o kernel.bin -T linker.ld multiboot_header.o kernel.o
```



```
readelf -a kernel.bin    # verify: entry point should be 0x100020
```

Figure 5 shows the `readelf` output confirming the entry point. The entry point is `0x100020` (not `0x100000`) because the 32-byte multiboot header occupies the first `0x20` bytes of the binary before the `start` symbol.


```

r4g4b@192:~/OS/M_P_1/Part2$ readelf -a kernel.bin
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:           ELF32
  Data:            2's complement, little endian
  Version:         1 (current)
  OS/ABI:          UNIX - System V
  ABI Version:     0
  Type:            EXEC (Executable file)
  Machine:         Intel 80386
  Version:         0x1
  Entry point address: 0x100020
  Start of program headers: 52 (bytes into file)
  Start of section headers: 4376 (bytes into file)
  Flags:           0x0
  Size of this header: 52 (bytes)
  Size of program headers: 32 (bytes)

```

Figure 5: `readelf -a kernel.bin`: entry point at 0x100020 (0x100000 base + 0x20 bytes for the multiboot header).

4.5 File 4 — grub.cfg

```
1 set timeout=10
2 set default=0
3
4 menuentry "My Mini OS" {
5     multiboot2 /boot/kernel.bin
6     boot
7 }
```

Listing 5: grub.cfg

4.6 Building the ISO Image

Set up the required directory tree, then run `grub2-mkrescue`:

```
mkdir -p cdrom/boot/grub
cp kernel.bin cdrom/boot/
cp grub.cfg cdrom/boot/grub/
grub2-mkrescue -o cdrom.iso cdrom
```

Required directory structure inside `cdrom/`:

```

1 cdrom/
2 +-- boot/
3     +-- kernel.bin
4     +-- grub/
5         +-- grub.cfg

```

Figure 6 shows the directory setup and Figure 7 confirms the ISO image was created successfully.

```

r4g4b@fedora:~/OS/M_P_1/Part2$ mkdir -p cdrom/boot/grub
r4g4b@fedora:~/OS/M_P_1/Part2$ ls
cdrom  grub.cfg  kernel.asm  kernel.bin  kernel.o  linker.ld  multiboot_header.asm  multiboot_header.o
r4g4b@fedora:~/OS/M_P_1/Part2$ cp kernel.bin cdrom/boot/
r4g4b@fedora:~/OS/M_P_1/Part2$ cp grub.cfg cdrom/boot/grub/
r4g4b@fedora:~/OS/M_P_1/Part2$ tree
.
├── cdrom
│   └── boot
│       ├── grub
│       │   └── grub.cfg
│       └── kernel.bin
├── grub.cfg
├── kernel.asm
├── kernel.bin
├── kernel.o
├── linker.ld
├── multiboot_header.asm
└── multiboot_header.o
4 directories, 9 files

```

Figure 6: Setting up the `cdrom/` directory structure and copying `kernel.bin` and `grub.cfg` to their required locations.

```

r4g4b@fedora:~/OS/M_P_1/Part2$ grub2-mkrescue -o cdrom.iso cdrom
xorriso 1.5.6 : RockRidge filesystem manipulator, libburnia project.

Drive current: -outdev 'stdio:cdrom.iso'
Media current: stdio file, overwriteable
Media status : is blank
Media summary: 0 sessions, 0 data blocks, 0 data, 51.7g free
Added to ISO image: directory '/='/tmp/grub.svXAuH'
xorriso : UPDATE : 976 files added in 1 seconds
Added to ISO image: directory '/='/home/r4g4b/OS/M_P_1/Part2/cdrom'
xorriso : UPDATE : 980 files added in 1 seconds
xorriso : NOTE : Copying to System Area: 512 bytes from file '/usr/lib/grub/i386-pc/boot_hybrid.img'
ISO image produced: 14635 sectors
Written to medium : 14635 sectors at LBA 0
Writing to 'stdio:cdrom.iso' completed successfully.

r4g4b@fedora:~/OS/M_P_1/Part2$ ls
cdrom  cdrom.iso  grub.cfg  kernel.asm  kernel.bin  kernel.o  linker.ld  multiboot_header.asm  multiboot_header.o
r4g4b@fedora:~/OS/M_P_1/Part2$

```

Figure 7: `grub2-mkrescue` successfully produces `cdrom.iso`.

4.7 Boot in QEMU

```
qemu-system-x86_64 -cdrom cdrom.iso
```

GRUB counts down 10 seconds, verifies the Multiboot 2 header in `kernel.bin`, loads it to `0x100000`, and transfers control to `start`. The single `mov` instruction writes OK to the VGA buffer; the CPU then halts.

4.8 Result

Figures 8–10 show the full boot sequence: the GRUB menu, the loading stage, and the final OK output.

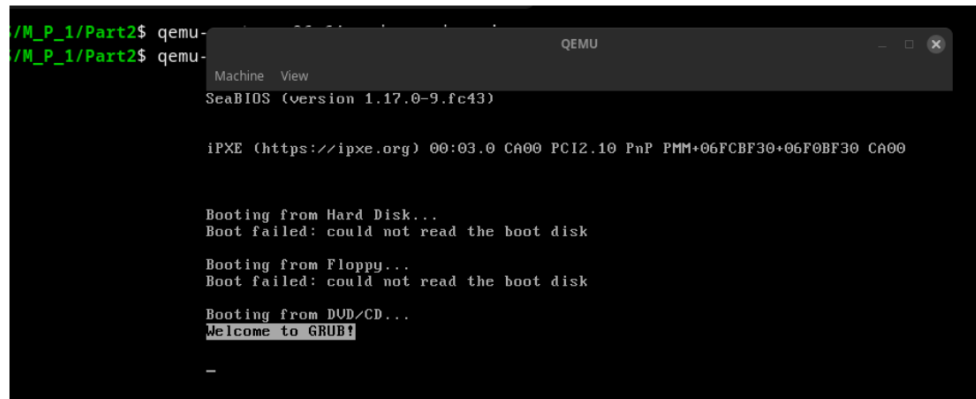


Figure 8: GRUB Info

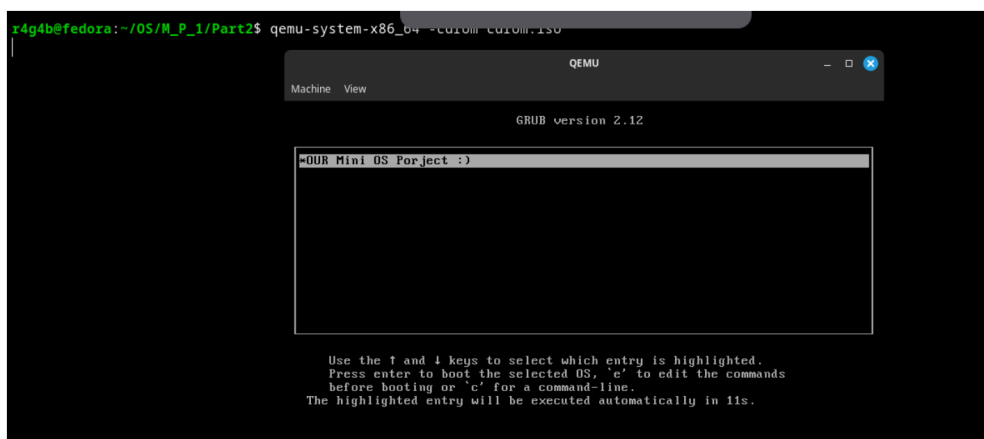


Figure 9: GRUB found My OS

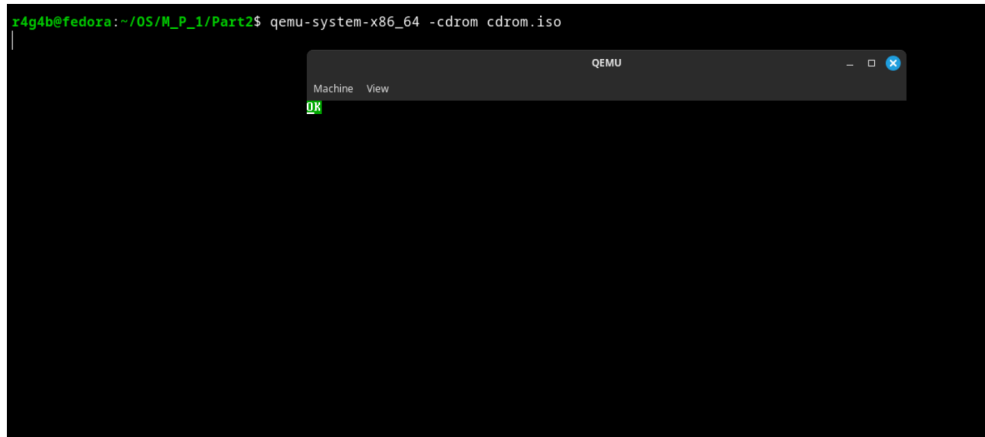


Figure 10: OK displayed in the top-left corner of the screen after the kernel gains control and writes to the VGA buffer.

5 Summary

	Part 1 — MBR	Part 2 — GRUB + Kernel
CPU mode	16-bit real mode	32-bit protected mode
Output method	BIOS interrupt 0x10	Direct VGA memory write
Boot medium	Raw 512-byte binary	ISO 9660 image
Loaded by	BIOS	GRUB (Multiboot 2)
Message	Hello	OK
Mode setup	Manual (segment registers)	Handled by GRUB

Part 1 demonstrates the absolute minimum the BIOS requires: a flat 512-byte binary with a magic signature, where every memory region must be configured by hand. Part 2 shows the approach used by real operating systems, where GRUB abstracts hardware initialisation so the kernel can assume a clean 32-bit environment from its very first instruction. Both parts confirm that the boot process is a reproducible sequence of well-defined conventions.

Acknowledgements

This project was completed in collaboration with Mosab Kamal (s-mosab.gabal@zewailcity.edu.eg) as part of the Operating Systems course at Zewail City of Science, Technology and Innovation.

A All Commands at a Glance

```
nasm -f bin -o boot.bin boot.asm
ls -l boot.bin
qemu-system-x86_64 -drive file=boot.bin,index=0,media=disk,format=raw
```

Listing 6: Part 1 — MBR

```
nasm -f elf32 multiboot_header.asm
nasm -f elf32 kernel.asm
ld -m elf_i386 -o kernel.bin -T linker.ld multiboot_header.o kernel.o
readelf -a kernel.bin

mkdir -p cdrom/boot/grub
cp kernel.bin cdrom/boot/
cp grub.cfg cdrom/boot/grub/

grub2-mkrescue -o cdrom.iso cdrom
```

```
qemu-system-x86_64 -cdrom cdrom.iso
```

Listing 7: Part 2 — GRUB + Kernel